

Building and Securing a REST API

Project Report

Course: Software Engineering — ALU

Team Members: Chol Mach · Kuol Akech · Abay Tessema · Alier Akuang

Repository: github.com/Chol-Mach-Kuol/Building-and-Securing-a-REST-API

1. Introduction to API Security

An Application Programming Interface (API) exposes data and functionality over a network. Without security controls, any client — including malicious ones — can read, modify or delete data. This project secures its API at the transport layer using HTTP Basic Authentication, which is the simplest standard mechanism available in Python's built-in `http.server` library.

The dataset used is a set of 1,691 real MTN MoMo SMS records parsed from `modified_sms_v2.xml`. Each record contains sensitive financial information: transaction amounts, sender and receiver names, phone numbers, and account balances. Protecting this data from unauthorised access is therefore not optional — it is a core requirement of the system.

Security in REST APIs typically operates at three levels:

- **Authentication** — confirming who the caller is (implemented in this project via Basic Auth).
- **Authorisation** — deciding what an authenticated caller is allowed to do (not implemented here; all authenticated users have full CRUD access).

- **Transport security** — encrypting data in transit so credentials and responses cannot be intercepted (requires HTTPS; not configured in this local development server).

2. API Endpoint Documentation

Base URL: `http://localhost:8000`

Authentication: HTTP Basic Auth — username `admin`, password `momo2024`

Format: All requests and responses use JSON.

2.1 GET /transactions

Returns all 1,691 transactions as a JSON array.

```
curl -u admin:momo2024 http://localhost:8000/transactions
```

Response 200 OK:

```
[
  {
    "id": 1,
    "transaction_id": "76662021700",
    "type": "incoming",
    "amount": 2000,
    "sender": "Jane Smith",
    "receiver": null,
    "fee": 0,
    "balance_after": 2000,
    "timestamp": "10 May 2024 4:30:58 PM"
  },
  ...
]
```

Error codes: 401 Unauthorized (wrong or missing credentials).

2.2 GET /transactions/{id}

Returns a single transaction by its numeric id.

```
curl -u admin:momo2024 http://localhost:8000/transactions/1
```

Response 200 OK: the transaction object shown above.

Error codes: 401 Unauthorized · 404 Not Found · 400 Bad Request (id is not a number).

2.3 POST /transactions

Creates a new transaction. The server assigns the id automatically.

```
curl -u admin:momo2024 -X POST http://localhost:8000/transactions \
  -H "Content-Type: application/json" \
  -d '{ "type": "payment", "amount": 5000, "sender": "self", "receiver": "Jane Smith",
      "fee": 100, "balance_after": 3000, "timestamp": "2024-05-10 10:00:00" } '
```

Response 201 Created with a `Location` header pointing to the new record.

Error codes: 401 · 400 (invalid JSON or negative amount).

2.4 PUT /transactions/{id}

Updates one or more fields of an existing transaction. Only the fields sent are changed.

```
curl -u admin:momo2024 -X PUT http://localhost:8000/transactions/1 \
  -H "Content-Type: application/json" \
  -d '{ "amount": 9999 } '
```

Response 200 OK: the full updated transaction.

Error codes: 401 · 404 Not Found · 400 Bad Request · 405 Method Not Allowed (PUT without an id).

2.5 DELETE /transactions/{id}

Permanently removes a transaction from the in-memory store.

```
curl -u admin:momo2024 -X DELETE
http://localhost:8000/transactions/1
```

Response 200 OK:

```
{ "message": "Transaction 1 deleted", "deleted": { ... } }
```

Error codes: 401 · 404 Not Found · 405 Method Not Allowed (DELETE without an id).

2.6 Error Code Summary

Code	Meaning
200 OK	Request succeeded (GET, PUT, DELETE)
201 Created	New transaction added (POST)
400 Bad Request	Invalid JSON, non-numeric id, or negative amount
401 Unauthorized	Credentials missing or wrong
404 Not Found	Transaction or endpoint does not exist
405 Method Not Allowed	Method does not match the URL pattern

3. DSA Integration — Linear Search vs Dictionary Lookup

The core DSA task was to compare two strategies for finding a transaction by its id, and to measure the difference in speed across different dataset sizes.

3.1 Linear Search — $O(n)$

The list of transactions is scanned from the beginning. Each record's id is compared to the target until a match is found or the list is exhausted. In the worst case (id not found), every record is checked.

3.2 Dictionary Lookup — $O(1)$ average

All transactions are stored in a Python dictionary keyed by id: `{id: transaction}`. Python computes a hash of the key and jumps directly to the correct slot, regardless of how many records exist.

3.3 Results (average time per single lookup)

Records	Linear Search	Dict Lookup	Binary Search	Dict Speed-up
20	0.273 μ s	0.054 μ s	0.314 μ s	5.1×
100	1.073 μ s	0.055 μ s	0.421 μ s	19.5×
500	4.951 μ s	0.053 μ s	0.606 μ s	93.7×
1,000	10.787 μ s	0.054 μ s	0.736 μ s	199.4×
1,691	17.442 μ s	0.055 μ s	0.890 μ s	314.5×

3.4 Worst-case Results (id does not exist)

Records	Linear Search	Dict Lookup
20	0.449 μ s	0.078 μ s
100	1.970 μ s	0.074 μ s
500	9.788 μ s	0.081 μ s
1,000	18.998 μ s	0.080 μ s
1,691	35.830 μ s	0.094 μ s

3.5 Reflection

Dictionary lookup is faster because Python's hash table resolves a key to a memory address in constant time, regardless of how many entries exist. Linear search time grows proportionally with the number of records — at 1,691 records it is $314.5\times$ slower than dictionary lookup.

Other data structures that could improve search efficiency:

- **Binary search on a sorted list** — $O(\log n)$; measured above. Slower than a dictionary for exact lookups but supports range queries (e.g. all transactions between two dates).
- **Balanced binary search tree (AVL / red-black tree)** — $O(\log n)$ for search, insert and delete while keeping data sorted.
- **B-tree index** (used by MySQL/PostgreSQL) — $O(\log n)$, optimised for large datasets stored on disk and efficient for both exact and range queries.

4. Basic Auth Limitations and Stronger Alternatives

4.1 Why Basic Auth is Weak

HTTP Basic Authentication encodes the username and password in Base64 and sends them in the `Authorization` header on every request:

```
Authorization: Basic YWRtaW46bW99tbzIwMjQ=
```

Base64 is not encryption — it is trivially reversible. Anyone who can intercept the HTTP traffic (for example, on an unsecured Wi-Fi network) can decode the header and obtain the credentials immediately. The specific weaknesses are:

- **No encryption** — credentials travel in plain text over HTTP.
- **Credentials sent on every request** — every API call exposes the password, increasing the attack surface.

- **No token expiry** — a stolen credential remains valid indefinitely until the password is changed manually.
- **Single shared credential** — all clients use the same username and password, so there is no way to revoke access for one client without affecting all others.
- **No audit trail** — it is impossible to distinguish which client made which request.

4.2 Stronger Alternatives

JSON Web Tokens (JWT): After a successful login, the server issues a signed token containing the user's identity and an expiry time. The client sends this token (not the password) on subsequent requests. The server verifies the signature without storing session state. Tokens expire automatically, limiting the damage if one is stolen.

OAuth 2.0: An industry-standard authorisation framework used by Google, GitHub and most major APIs. It separates authentication (proving identity) from authorisation (granting specific permissions). Clients receive short-lived access tokens and long-lived refresh tokens. OAuth 2.0 also supports scopes, so a token can be restricted to read-only access, for example.

API Keys with HTTPS: A unique key is issued to each client. Keys can be revoked individually, rotated on a schedule, and logged per request. This is simpler than OAuth 2.0 and appropriate for server-to-server communication where a human login flow is not needed.

For a production deployment of this API, the recommended approach would be JWT over HTTPS: it eliminates the need to send the password on every request, tokens expire automatically, and the implementation overhead is manageable with standard Python libraries such as `PyJWT`.

End of Report